SESIÓN 05: FRAGMENTACIÓN POR REGIONES Y CARGA MASIVA

Particiones · Índices · ETL

Qué se practica

Cuando el volumen crece, una tabla única se vuelve lenta. Aquí se usan tres técnicas:

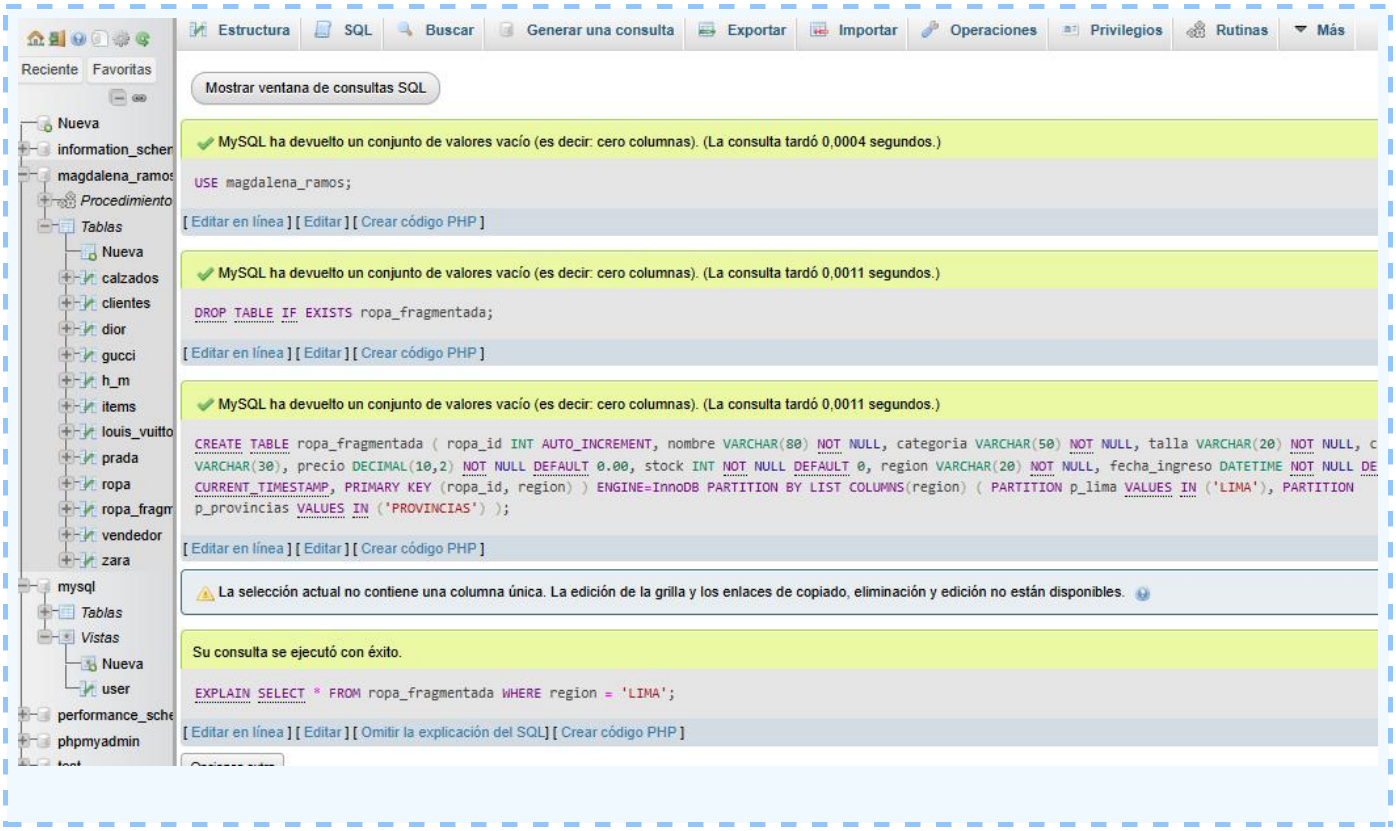
- Particionado por región: el motor puede ignorar fragmentos irrelevantes (pruning).
- Índices compuestos: evitar escaneo completo (type: ALL).
- Carga masiva (ETL): insertar miles de filas de una vez.

Paso 1 · Tabla fragmentada por región

Se crea ropa_fragmentada con particiones LIST (LIMA / PROVINCIAS). La PK incluye region porque la columna de partición debe formar parte de ella.

```
USE magdalena_ramos;
DROP TABLE IF EXISTS ropa_fragmentada;
CREATE TABLE ropa_fragmentada (
  ropa_id INT AUTO_INCREMENT,
  nombre VARCHAR(80) NOT NULL,
  categoria VARCHAR(50) NOT NULL,
  talla VARCHAR(20) NOT NULL,
  color VARCHAR(30),
  precio DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  stock INT NOT NULL DEFAULT 0,
  region VARCHAR(20) NOT NULL,
  fecha_ingreso DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (ropa_id, region)
) ENGINE=InnoDB PARTITION BY LIST COLUMNS(region) (
  PARTITION p_lima VALUES IN ('LIMA'),
  PARTITION p_provincias VALUES IN ('PROVINCIAS')
);

EXPLAIN SELECT * FROM ropa_fragmentada WHERE region = 'LIMA';
```



Qué hace: reparte filas en dos fragmentos. Un WHERE region='LIMA' solo toca p_lima.

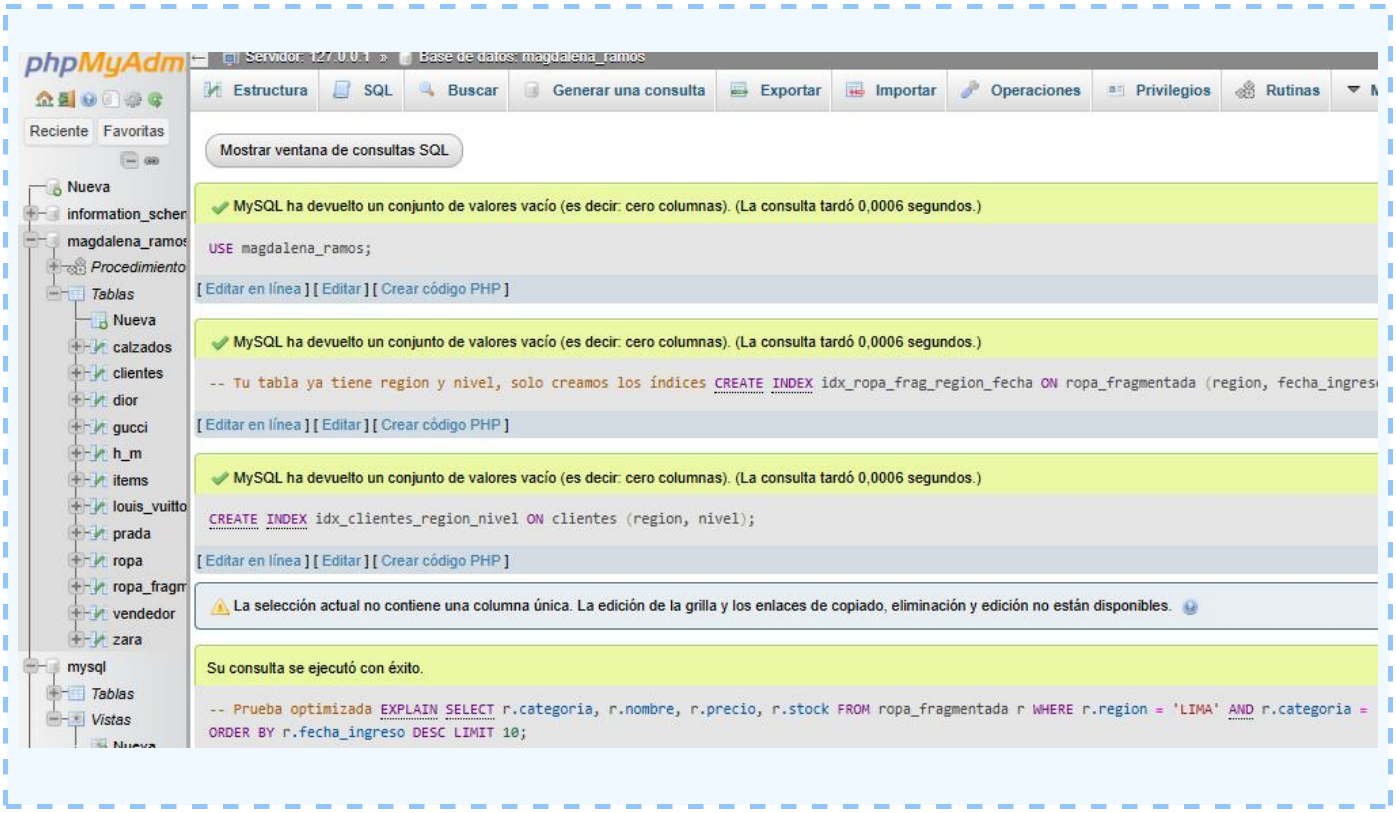
Paso 2 · Índices de alto rendimiento

Sin índice, EXPLAIN suele mostrar type: ALL. Con índices compuestos, aparece key y menos filas examinadas.

```
USE magdalena_ramos;

-- Tu tabla ya tiene region y nivel, solo creamos los índices
CREATE INDEX idx_ropa_frag_region_fecha ON ropa_fragmentada (region, fecha_ingreso DESC);
CREATE INDEX idx_clientes_region_nivel ON clientes (region, nivel);

-- Prueba optimizada
EXPLAIN SELECT r.categoria, r.nombre, r.precio, r.stock
FROM ropa_fragmentada r
WHERE r.region = 'LIMA' AND r.categoria = 'Vestidos'
ORDER BY r.fecha_ingreso DESC LIMIT 10;
```



Qué mirar: columna key con el nombre del índice y type distinto de ALL.

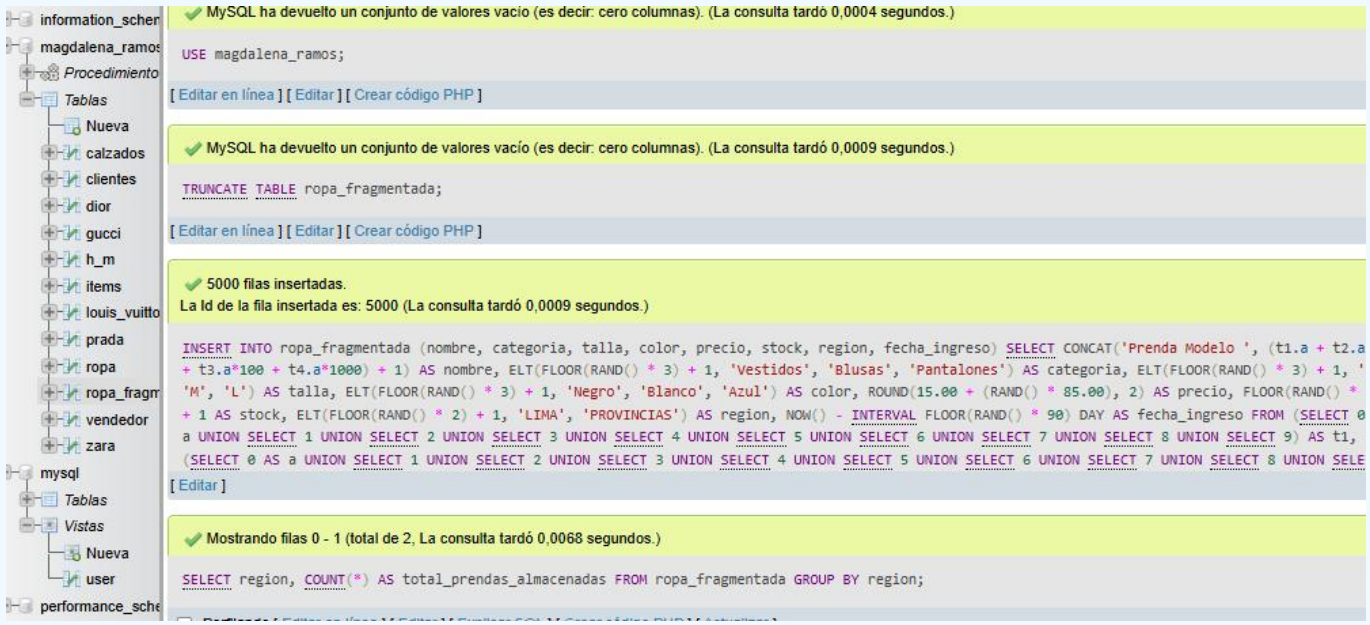
Paso 3 · Carga masiva tipo ETL

Se generan miles de prendas de prueba entre LIMA y PROVINCIAS. En producción equivaldría a LOAD DATA INFILE o Importar CSV en phpMyAdmin.

```
USE magdalena_ramos;
TRUNCATE TABLE ropa_fragmentada;

INSERT INTO ropa_fragmentada (nombre, categoria, talla, color, precio, stock, region, fecha_ingreso)
SELECT
  CONCAT('Prenda Modelo ', (t1.a + t2.a*10 + t3.a*100 + t4.a*1000) + 1) AS nombre,
  ELT(FLOOR(RAND() * 3) + 1, 'Vestidos', 'Blusas', 'Pantalones') AS categoria,
  ELT(FLOOR(RAND() * 3) + 1, 'S', 'M', 'L') AS talla,
  ELT(FLOOR(RAND() * 3) + 1, 'Negro', 'Blanco', 'Azul') AS color,
  ROUND(15.00 + (RAND() * 85.00), 2) AS precio,
  FLOOR(RAND() * 50) + 1 AS stock,
  ELT(FLOOR(RAND() * 2) + 1, 'LIMA', 'PROVINCIAS') AS region,
  NOW() - INTERVAL FLOOR(RAND() * 90) DAY AS fecha_ingreso
FROM
 (SELECT 0 AS a UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT 5 UNION SELECT 6
UNION SELECT 7 UNION SELECT 8 UNION SELECT 9) AS t1,
 (SELECT 0 AS a UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT 5 UNION SELECT 6
UNION SELECT 7 UNION SELECT 8 UNION SELECT 9) AS t2,
 (SELECT 0 AS a UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT 5 UNION SELECT 6
UNION SELECT 7 UNION SELECT 8 UNION SELECT 9) AS t3,
 (SELECT 0 AS a UNION SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4) AS t4;
```

```
SELECT region, COUNT(*) AS total_prendas_almacenadas
FROM ropa_fragmentada
GROUP BY region;
```



Resultado esperado: miles de filas repartidas entre LIMA y PROVINCIAS.

Conclusión de la sesión

Particionar por un criterio natural acelera consultas filtradas. Los índices evitan escaneos completos. La carga masiva muestra cómo poblar volúmenes grandes sin insertar fila a fila. Juntas preparan la base para crecer sin degradar el tiempo de respuesta.